BUILD YOUR OWN PRIVATE COSMOS BLOCKCHAIN

I. Prerequisites

- Familiarity with blockchain concepts
- Installed tools:
  - Git
  - Python
  - Docker
  - Go
  - Node.js
  - Rust

II. Set up private blockchain with Ignite CLI

1. Install Ignite CLI

This entire tutorial was built using the Ignite CLI version 0.22.1. To install it at the command line:

```
~$ curl https://get.ignite.com/cli@v0.22.1! | bash
```

You can verify the version of Ignite CLI you have once it is installed:

```
$ ignite version
```

This prints its version:

```
Ignite CLI version:     v0.22.1
...
```

2. Prepare Docker

If you want to allow for portability and avoid version issues, it is advisable to use Docker (opens new window). If you are new to Docker, have a look at this tutorial.

First, you need to create a Dockerfile that details the same preparation steps. Save this as Dockerfile-ubuntu:

```
FROM --platform=linux ubuntu:22.04
ARG BUILDARCH

# Change your versions here
ENV GO_VERSION=1.18.3
ENV IGNITE_VERSION=0.22.1
ENV NODE_VERSION=18.x

ENV LOCAL=/usr/local
ENV GOROOT=$LOCAL/go
ENV HOME=/root
ENV GOPATH=$HOME/go
```

```
ENV PATH=$GOROOT/bin:$GOPATH/bin:$PATH

RUN mkdir -p $GOPATH/bin

ENV PACKAGES curl gcc
RUN apt-get update
RUN apt-get install -y $PACKAGES

# Install Go
RUN curl -L https://go.dev/dl/go${GO_VERSION}.linux-
$BUILDARCH.tar.gz | tar -C $LOCAL -xzf -

# Install ignite
RUN curl -L https://get.ignite.com/cli@v${IGNITE_VERSION}! |
bash

RUN apt-get clean

EXPOSE 1317 3000 4500 5000 26657

WORKDIR /checkers
```

Note that the linked code contains more lines than shown above. Because you do not yet have a go.mod file, stick to the lines in the quoted code above for now.

Next you need to create the Docker image:

```
$ docker build -f Dockerfile-ubuntu . -t checkers_i
```

You can confirm the installed version of Ignite:

```
$ docker run --rm -it checkers_i ignite version
```

It should return, among other things:

```
Ignite CLI version:     v0.22.1
```

That is the bare minimum required to be able to run the commands that come on this page. If at a later stage you want to create a persistent container named checkers, you can do:

```
$ docker create --name checkers -i \
    -v $(pwd):/checkers -w /checkers \
    -p 1317:1317 -p 3000:3000 -p 4500:4500 -p 5000:5000 -p
26657:26657 \
    checkers_i
$ docker start checkers
```

3. Your chain

Start by scaffolding a basic chain called checkers that you will place under the GitHub path alice:

```
$ ignite scaffold chain github.com/alice/checkers
```

github.com/alice/checkers is the name of the Golang module by which this project will be known. If you own the github.com/alice path, you can even eventually host it there and have other people use your project as a module.

The scaffolding takes some time as it generates the source code for a fully functional, ready-to-use blockchain. Ignite CLI creates a folder named checkers and scaffolds the chain inside it.

The checkers folder contains several generated files and directories that make up the structure of a Cosmos SDK blockchain. It contains the following folders:

- app (opens new window): a folder for the application.
- cmd (opens new window): a folder for the command-line interface commands.
- proto (opens new window): a folder for the Protobuf objects definitions.
- vue (opens new window): a folder for the auto-generated UI.
- x (opens new window): a folder for all your modules, in particular checkers.

Go to the checkers folder and run:

```
$ cd checkers
$ ignite chain serve
```

The ignite chain serve command downloads (a lot of) dependencies and compiles the source code into a binary called checkersd. This command:

- Installs all dependencies.
- Builds Protobuf files.
- Compiles the application.
- Initializes the node with a single validator.
- Adds accounts.

After the chain serve command completes, you have a local testnet with a running node. What about the added accounts? Take a look:

```
accounts:
  - name: alice
    coins: ["20000token", "200000000stake"]
  - name: bob
    coins: ["10000token", "100000000stake"]
validator:
  name: alice
  staked: "100000000stake"
client:
  openapi:
    path: "docs/static/openapi.yml"
  vuex:
    path: "vue/src/store"
faucet:
  name: bob
  coins: ["5token", "100000stake"]
```

In this file you can set the accounts, the accounts' starting balances, and the validator. You can also let Ignite CLI generate a client and a faucet. The faucet gives away five token and 100,000 stake tokens belonging to Bob each time it is called.

You can observe the endpoints of the blockchain in the output of the ignite chain serve command:

```
🌍 Tendermint node: http://0.0.0.0:26657
🌍 Blockchain API: http://0.0.0.0:1317
🌍 Token faucet: http://0.0.0.0:4500
```

4. Interact via the CLI

You can already interact with your running chain. With the chain running in its shell, open another shell and try:

```
$ checkersd status
```

This prints:


```
{"NodeInfo":{"protocol_version":{"p2p":"8","block":"11","app":"0
"},"id":"8df1253b4deb59f63cc912dce096ab010c951e9d","listen_addr"
:"tcp://0.0.0.0:26656","network":"checkers","version":"0.34.19",
"channels":"40202122233038606100","moniker":"mynode","other":{"t
x_index":"on","rpc_address":"tcp://0.0.0.0:26657"}},"SyncInfo":{
"latest_block_hash":"6F167C4E2C99385857663B9531016DBC85DC0AEC1B5
8BF759B729EEAC843B92A","latest_app_hash":"EE408C7580E1E4A81E2019
0D9131FBD07AE1C536D3507DF9C6E0CB476A2D7680","latest_block_height
":"13","latest_block_time":"2022-06-
27T15:43:14.906782552Z","earliest_block_hash":"48250CF257E117F28
FE207A71DDCA67459FEBE2EF1367D7B0EAE43754D5A53A1","earliest_app_h
ash":"E3B0C44298FC1C149AFBF4C8996FB92427AE41E4649B934CA495991B78
52B855","earliest_block_height":"1","earliest_block_time":"2022-
06-
27T15:42:57.697745314Z","catching_up":false},"ValidatorInfo":{"A
ddress":"98E9E157C87A44503BB8D01CAFC97DDB5D0C78DE","PubKey":{"ty
pe":"tendermint/PubKeyEd25519","value":"U3wlX8+lx6YQq3g2QbYnnAdU
uMQ7AlMXH21Vxrq2OHg="},"VotingPower":"100"}}
```


In there you can see a hint of liveness: "latest_block_height":"13". You can use this one-liner to better see the information:

```
$ checkersd status 2>&1 | jq
```

You can learn a lot by going through the possibilities with:

```
$ checkersd --help
$ checkersd status --help
$ checkersd query --help
```

And so on.